DATA305 / DATA475 · Machine Learning Methods

Transfer Learning & Fine-Tuning

Pretrained Models · Feature Extraction · Domain Adaptation

Reuse, don't rebuild — but reuse it correctly

Lecture 9 Week 8 · Google Colab ready

Binh Nguyen · School of Mathematics and Statistics · Victoria University of Wellington

Today's Agenda & Learning Objectives

Lecture 9

Five sections · flower_photos dataset (3,670 images, 5 classes) · EfficientNetB0 backbone

01

Why Transfer Learning Works

Visual hierarchy, ImageNet → your task.

02

Feature Extraction

Freeze the base, train the head only.

03

Fine-Tuning Correctly

Unfreeze top-N, tiny LR, recompile.

04

Domain Decisions

Domain × dataset size → strategy choice.

05

Colab Experiment

Scratch vs feature extr. vs fine-tuning.

Learning Objectives

✓ Apply feature extraction with a pretrained model.

✓ Choose unfreezing schedule from data + domain.

✓ Quantify accuracy vs training-time trade-off.

Dataset: flower_photos via tf.keras.utils.get_file · Géron Ch. 14, pp. 621–640

Lecture 8 → Lecture 9

Bridge

In L8 we mapped the model zoo. Today we learn how to actually use pretrained models.

Lecture 8 — Covered

- ✓ Historical arc: AlexNet → VGG → ResNet → EfficientNet
- ✓ Residual connections, depthwise separable convolutions
- ✓ Compound scaling: EfficientNet B0–B7
- ✓ keras.applications: loading pretrained weights
- ✓ Feature extraction vs fine-tuning — preview only

Lecture 9 — Today

- → Why pretrained features transfer across domains
- → Feature extraction workflow in Keras — step by step
- → Fine-tuning: progressive unfreezing and LR strategy
- → BatchNorm caveat during fine-tuning
- → Domain similarity → strategy decision matrix
- → Three-way comparison on flower_photos (Models A / B / C)

« Reuse, don't rebuild — but reuse it correctly »

Today's running example: flower_photos with EfficientNetB0 · full Colab notebook provided

01

Why Transfer Learning Works

Visual hierarchy · ImageNet → your task · two strategies at a glance

Why Transfer Learning Works

Section 01

A network trained on 1.2 M images has already learned to see.

ImageNet pretraining learns reusable low- and mid-level visual features.

Early layers are general (edges, textures); top layers are increasingly task-specific.

Transfer learning works because many vision tasks share the same building blocks.

→ Reuse the backbone; replace only the part that is specific to your task.

Early layers

Edges, color gradients, corners

Middle layers

Textures, repeated motifs, parts

Deeper layers

Object-level patterns (eyes, petals…)

Top layers

Task-specific — replace these!

Early layers generalize most; top layers specialize to the original task

From ImageNet to Your Task

Section 01

The domain gap determines how much of the backbone you can trust.

Gap		
Near	**ImageNet → flower_photos** Natural-photo domain — close to ImageNet. Edges and textures transfer directly.	✓ Feature extraction first
Moderate	**ImageNet → Medical X-rays** Shared low-level cues but a different modality. Start with feature extraction, then likely fine-tune top layers.	Fine-tune top layers
Far	**ImageNet → Satellite imagery** Very different statistics. High-level features differ; domain-specific pretraining may help more than standard fine-tuning.	Deep fine-tune; domain pretraining; or scratch if enough data

Rule of thumb (heuristic, not strict): the closer the domains, the less fine-tuning you need

Two Strategies at a Glance

Section 01

Choose your starting point before writing any code.

Dimension	Feature Extraction	Fine-Tuning
Trainable params	Head only (small fraction)	Head + selected top layers (larger fraction)
Training speed	Very fast (few epochs)	Slower — more passes needed
Data needed	Works with small datasets	Needs more data to avoid overfit
Overfit risk	Low	Medium — depends on LR and unfreeze depth
Expected gain	Strong baseline quickly	Can surpass feature extraction if done right
When to start	Always — run this first	Only after the head has converged

Start with feature extraction → measure val_accuracy → decide whether to fine-tune

02

Feature Extraction

Freeze the backbone · train the head only · fast and stable baseline

Feature Extraction — The Idea

Section 02

Treat the pretrained backbone as a fixed feature extractor.

Pretrained Backbone

`base.trainable = False`

Weights locked

→

Global Average Pooling

Spatial → vector

No parameters

→

Dense Head

`Dense(256, relu)`

`Dropout(0.3)`

`Dense(5, softmax)`

Fast

Only the head trains — faster than full fine-tuning; practical even on CPU.

Stable

No risk of corrupting pretrained weights — strong baseline by design.

Effective

Often gets close to fine-tuning on small, near-domain datasets with much less effort.

base.trainable = False prevents gradient updates from reaching the backbone during backprop

Feature Extraction — Keras Workflow

Section 02

Four steps: load backbone → freeze → attach head → compile.

```python
import tensorflow as tf
from tensorflow.keras import layers

# 1. Load pretrained base — no top classifier
base = tf.keras.applications.EfficientNetB0(
    include_top=False, weights="imagenet",
    input_shape=(224, 224, 3))

# 2. Freeze all backbone layers
base.trainable = False

# 3. Build head on top
inputs  = tf.keras.Input(shape=(224, 224, 3))
x       = base(inputs, training=False)
x       = layers.GlobalAveragePooling2D()(x)
x       = layers.Dense(256, activation="relu")(x)
x       = layers.Dropout(0.3)(x)
outputs = layers.Dense(5, activation="softmax")(x)
model   = tf.keras.Model(inputs, outputs)

# 4. Compile (callbacks + fit on the next code slide)
model.compile(optimizer=tf.keras.optimizers.Adam(1e-3),
              loss="sparse_categorical_crossentropy",
              metrics=["accuracy"])
```

Step 1

include_top=False drops the ImageNet head. We attach our own.

Step 2

base.trainable=False locks all 237 base layers — none update.

Step 3

training=False on the base call keeps BatchNorm using running stats.

Step 4

LR = 1e-3 is fine when only training the small head (~330K params).

Notebook §3.1–§3.2 · abbreviated: callbacks (EarlyStopping, ModelCheckpoint) added on next code slide

Why GAP + Small Head?

Section 02

Design choice: Global Average Pooling vs Flatten + large Dense.

✓ GAP + Dense(256) [recommended]

- Feature map (7×7×1280) → mean across spatial → 1280-vector
- Dense(256) adds only ~328 K parameters
- Fewer params = less overfitting on small datasets
- Spatial invariance built in — no positional bias
- Standard practice in modern transfer-learning recipes

✗ Flatten + Dense(1024) [not recommended]

- Feature map (7×7×1280) → 62,720-vector via Flatten
- Dense(1024) adds ~64 M params — often larger than the base!
- Massive overfit risk on a 3,670-image dataset
- Positional sensitivity: spatial information encoded in index
- Slow to train; rarely outperforms GAP here

GAP + small Dense head: fewer parameters, better regularization, standard practice for transfer learning.

EfficientNetB0 output before GAP: 7×7×1,280 = 62,720 values. GAP collapses to 1,280

03

Fine-Tuning Correctly

Progressive unfreezing · learning-rate rule · BatchNorm caveat

Fine-Tuning — The Idea

Section 03

After the head converges, selectively unfreeze top layers to adapt deeper features.

- Head-only training reaches a plateau — the fixed backbone features may not be perfectly aligned with the target domain.
- Fine-tuning lets the top layers adjust their representations to the new task.
- Prerequisite: the head must already be stable (low val_loss, no wild swings).
- Use a very small learning rate — typically 10–100× lower than the head-training LR.
- Only unfreeze the top few blocks initially — don't touch the deep base unless necessary.

Trainable Regions

- Base — early layers (frozen)
- Base — middle layers (frozen)
- **Base — top blocks (unfreeze)**
- GlobalAveragePooling2D
- **Dense head (trained)**

Prerequisite: run feature extraction first and wait for convergence before unfreezing

Fine-Tuning ≠ "Unfreeze Everything Immediately"

Section 03

The most common fine-tuning mistake — and why it fails.

✗ What happens when you unfreeze all at once

- Pretrained weights receive large gradient updates from a random head.
- Low-level features (edges, textures) get corrupted — catastrophic forgetting.
- High LR + wide unfreeze → loss spikes or diverges.
- Even if loss recovers, you end up far from the pretrained minimum.
- You have effectively thrown away the ImageNet training — no benefit.

✓ Correct approach: phase by phase

- Phase 1: train head only until val_loss stabilizes (5–15 epochs).
- Phase 2: unfreeze only the top-N layers; use LR 10–100× smaller.
- Phase 3 (optional): if still underfit, unfreeze more blocks — one at a time.
- Monitor val_loss continuously — stop if it rises after unfreezing.
- Recompile the model after every change to trainable flags.

Catastrophic forgetting: rapid overwriting of useful pretrained representations due to too-large gradients

Progressive Fine-Tuning — Three Phases

Section 03

Always recompile after changing trainable flags — the optimizer state must reset.

Phase 1

Head Only

Freeze all base layers. Train only GAP + Dense head from random init. Let head reach low val_loss before touching the backbone.

LR = 1e-3 · ~10 epochs

```
base.trainable = False
model.compile(Adam(1e-3), ...)
model.fit(train_ds, epochs=10)
```

Phase 2

Unfreeze Top-N

Unfreeze the top layers of the backbone (e.g. top-30 for EfficientNetB0). Re-compile with a ~100× smaller LR. Exact depth depends on architecture.

LR = 1e-5 · ~10 more epochs

```
base.trainable = True
for l in base.layers[:-30]:
    l.trainable = False
model.compile(Adam(1e-5), ...)
```

Phase 3

Deeper Unfreeze (optional)

Only if performance still lags. Unfreeze more blocks at an even lower LR. Risk of overfitting grows. This step is optional.

LR = 1e-5 to 1e-6 · 5–10 more epochs

```
for l in base.layers[:-60]:
    l.trainable = False
model.compile(Adam(1e-6), ...)
```

Always re-compile after changing trainable flags — the optimizer's state must reset

Learning Rate Rule for Fine-Tuning

Section 03

The right LR is often the difference between successful fine-tuning and catastrophic forgetting.

Head training *(feature extraction)*	Shallow fine-tune *(top ~30 layers)*	Deep fine-tune *(more layers)*
1e-3	**1e-4 to 1e-5**	**1e-5 to 1e-6**
×1 relative to head LR	÷10 to ÷100 relative to head LR	÷100 to ÷1000 relative to head LR
Safe starting point	*Standard fine-tune*	*Deep adaptation*

Why so small?

Pretrained weights are already near a good optimum. A large step throws you away from that optimum. A small step nudges the representations toward the target domain without destroying what ImageNet training found.

If val_loss rises right after unfreezing → LR may be too large, or too many layers unfrozen

BatchNorm — Special Care Required

Section 03

The most overlooked pitfall in transfer learning.

What BatchNorm stores

Each BN layer has two kinds of state. **Trainable parameters:** γ (scale) and β (shift) — updated by gradients.

Non-trainable state: running_mean and running_variance — updated only during forward passes in *training mode*. They do NOT change when base.trainable=True unless you call base(x, training=True).

Feature extraction (correct)

```
x = base(inputs, training=False)
```

BN uses stored ImageNet stats — stable, recommended for this course.

Fine-tuning: BN still frozen (safest)

```
x = base(inputs, training=False)
```

Even after base.trainable=True. Backbone γ/β can update, but BN running stats stay fixed. Usually sufficient.

Fine-tuning: BN updating (risky)

```
x = base(inputs, training=True)
```

BN running stats adapt to your domain. Can work, but needs large batch, careful LR, and close monitoring.

Practical default for this course: keep base in inference mode (training=False) during transfer learning

04

Domain Decisions

Domain similarity × dataset size → strategy choice · decision flowchart

Domain Similarity & Strategy Matrix

Section 04

Match your strategy to your data — not to the latest paper.

	Near domain	Moderate gap	Far domain
Small dataset *(< 2K images)*	Feature extraction	Feature extraction → tune top layers	Fine-tune top few blocks
Medium dataset *(2K–20K)*	Feature extract + careful fine-tune	Unfreeze top layers low LR	Deep fine-tune or scratch*
Large dataset *(> 20K)*	Unfreeze top layers low LR	Deep fine-tune most backbone	Deep fine-tune; scratch if enough data

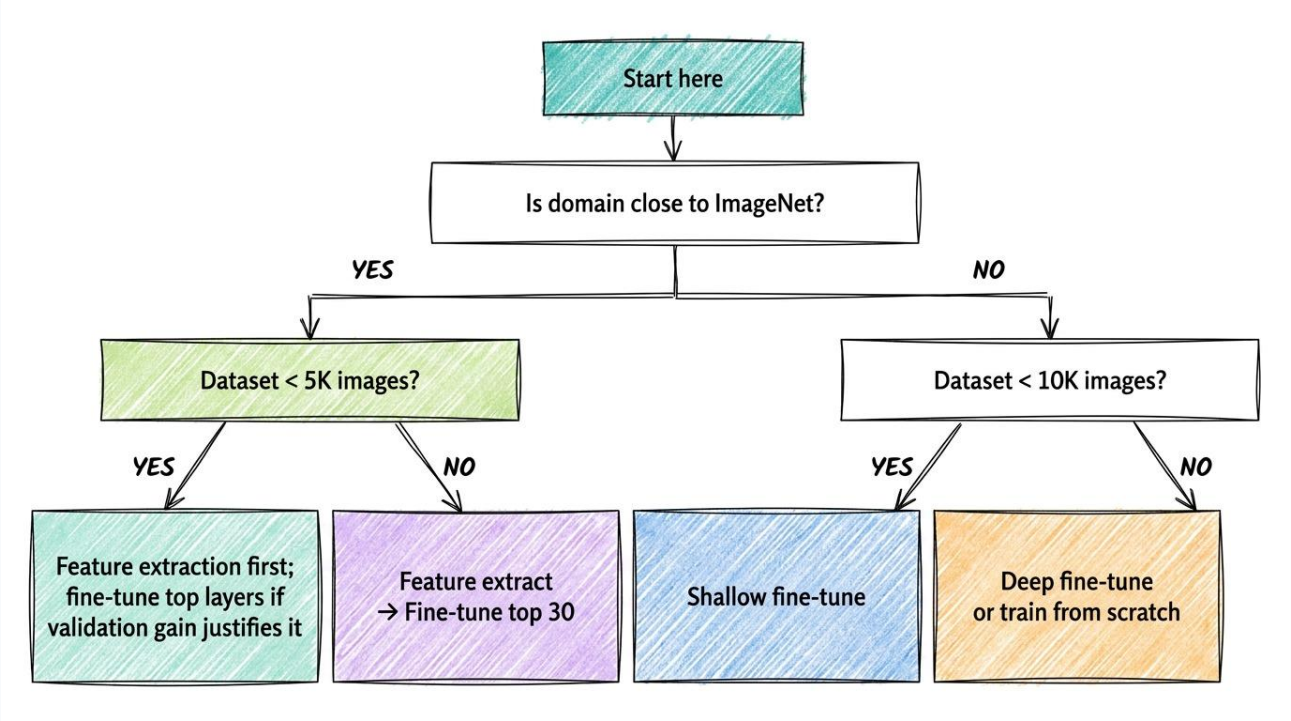
flower_photos = 3,670 images (medium dataset, near domain) → feature extraction first; fine-tune if validation gain justifies

Thresholds are heuristic, not strict — the right choice also depends on compute budget and domain quirks

Strategy Decision Guide

Section 04

A practical flowchart for choosing your transfer-learning approach.



flower_photos: domain near, dataset medium → feature extraction path · also weigh compute budget and turnaround time

05

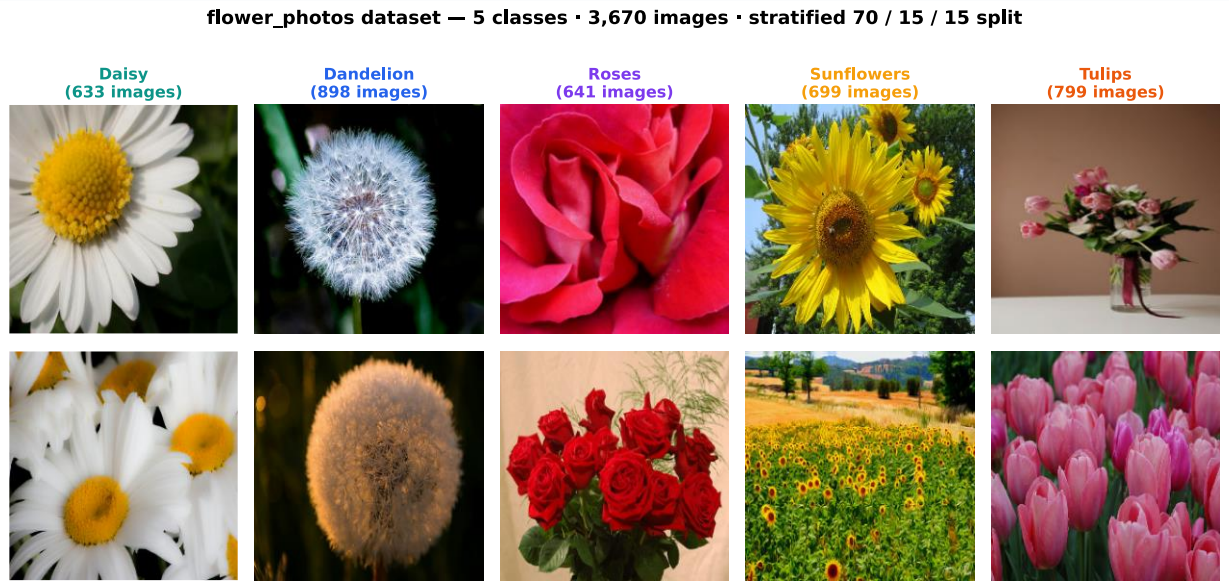
Colab Experiment & Results

Three models on flower_photos · measured accuracy and training time

Dataset Snapshot — flower_photos

Section 05

3,670 images · 5 classes · natural-photo domain · stratified 70 / 15 / 15 split.



Mild imbalance, no class weighting needed.

Total images

3,670

Classes

5

Train / Val / Test

70 / 15 / 15 %

Stratified by class
Fixed random seed

Data Pipeline — flower_photos

Section 05

Resize → cache → shuffle → augment (train only) → batch → prefetch.

Resize 224×224 → Cast float32 → Cache → Shuffle (train only) → Augment (train only) → Batch + Prefetch

```
augment = tf.keras.Sequential([
    layers.RandomFlip("horizontal"),
    layers.RandomRotation(0.12),
    layers.RandomZoom(0.12),
    layers.RandomContrast(0.10),
])

def cast_float(img, label):
    return tf.cast(img, tf.float32), label   # keep [0,255]

train_ds = (train_raw
    .map(cast_float, num_parallel_calls=AUTOTUNE)
    .cache().shuffle(1000, seed=SEED)
    .map(lambda x, y: (augment(x), y), num_parallel_calls=AUTOTUNE)
    .batch(BATCH_SIZE).prefetch(AUTOTUNE))
```

IMG_SIZE = 224

EfficientNetB0 was pretrained at 224×224. Wrong size → mismatched feature maps.

No preprocess_input()

EfficientNetB0 has built-in rescaling. Pass float32 [0,255]. Other backbones differ — match the family.

No augment in val/test

Random augmentation only on the training pipeline — val/test stay deterministic.

Notebook §1.5 · cache() before shuffle keeps deterministic preprocessing fast; suitable when dataset fits in memory

Colab Experiment Design

Section 05

Same dataset · same split · same augmentation · three strategies.

Model A
Train from Scratch

Small custom CNN (3 conv blocks). No pretrained weights. Baseline to beat.

Architecture	`3× Conv2D + MaxPool`
Head	`Dense(128) + Dropout(0.4)`
Optimizer	`Adam 1e-3`
Total params	`~111 K`

Model B
Feature Extraction

EfficientNetB0 frozen. Train only GAP + Dense head. flower_photos, 5 classes.

Backbone	`base.trainable = False`
Head	`GAP → Dense(256) → Dense(5)`
Optimizer	`Adam 1e-3`
Trainable	`4.38 M total · 329 K trainable`

Model C
Progressive Fine-Tuning

Start from best Model B. Unfreeze top-30 backbone layers, LR=1e-5, 20 more epochs.

Phase 1	`head-only ≤20 ep`
Phase 2	`top-30 unfreeze ≤20 ep`
Optimizer	`Adam 1e-5 (phase 2)`
Trainable	`4.38 M total · 1.81 M trainable`

Model A is an instructional baseline — not a tuned SOTA. Shared: same 70/15/15 split, augmentation, hardware.

Code — Feature Extraction (Model B)

Code

Frozen backbone + small head + EarlyStopping & ModelCheckpoint.

```
# — Model B: EfficientNetB0 Feature Extraction —
base_B = tf.keras.applications.EfficientNetB0(
    include_top=False, weights="imagenet",
    input_shape=(224, 224, 3))
base_B.trainable = False                  # all backbone layers frozen

inputs  = tf.keras.Input(shape=(224, 224, 3))
x       = base_B(inputs, training=False)  # BN in inference mode
x       = layers.GlobalAveragePooling2D()(x)
x       = layers.Dense(256, activation="relu")(x)
x       = layers.Dropout(0.3)(x)
outputs = layers.Dense(5, activation="softmax")(x)
model_B = tf.keras.Model(inputs, outputs)

model_B.compile(optimizer=optimizers.Adam(1e-3),
                loss="sparse_categorical_crossentropy",
                metrics=["accuracy"])

cb_B = [callbacks.EarlyStopping(patience=5, monitor="val_accuracy",
                                restore_best_weights=True),
        callbacks.ModelCheckpoint("best_model_B.keras",
                                  save_best_only=True)]
```

training=False on base

Keeps BatchNorm in inference mode — uses ImageNet running stats.

Callbacks

EarlyStopping(patience=5) + ModelCheckpoint save the best epoch on val_accuracy.

LR = 1e-3

Appropriate here — only ~329K head parameters update. Backbone is frozen.

Expected on Colab T4

Strong baseline within 5–10 epochs. Varies with split, seed, and callbacks.

Notebook §3.1–§3.2 · patience=5 / epochs=20 match the notebook configuration

Code — Fine-Tuning Top-30 Layers (Model C)

Code

Load best B · unfreeze top-30 · freeze BN · recompile · fine-tune at LR = 1e-5.

```
# — Model C: Progressive Fine-Tuning —
# Phase 1: load best feature-extraction weights
model_C = tf.keras.models.load_model("best_model_B.keras")
base_C  = model_C.get_layer("efficientnetb0")    # access by name

# Phase 2: unfreeze top 30 layers of the backbone
base_C.trainable = True
for layer in base_C.layers[:-30]:
    layer.trainable = False

# Freeze BatchNorm layers explicitly — keep running stats stable
for layer in base_C.layers:
    if isinstance(layer, tf.keras.layers.BatchNormalization):
        layer.trainable = False

# MUST recompile after changing trainable flags
model_C.compile(optimizer=optimizers.Adam(1e-5),  # 100× smaller LR
                loss="sparse_categorical_crossentropy",
                metrics=["accuracy"])

cb_C = [callbacks.EarlyStopping(patience=6, restore_best_weights=True),
        callbacks.ModelCheckpoint("best_model_C.keras", save_best_only=True),
        callbacks.ReduceLROnPlateau(factor=0.5, patience=3, min_lr=1e-7)]
```

Access by name

get_layer("efficientnetb0") is robust — don't index by position.

Freeze BN explicitly

Even when base.trainable=True, keep BN running stats fixed.

Always recompile

After every change to trainable flags — optimizer's moment estimates must reset.

LR = 1e-5

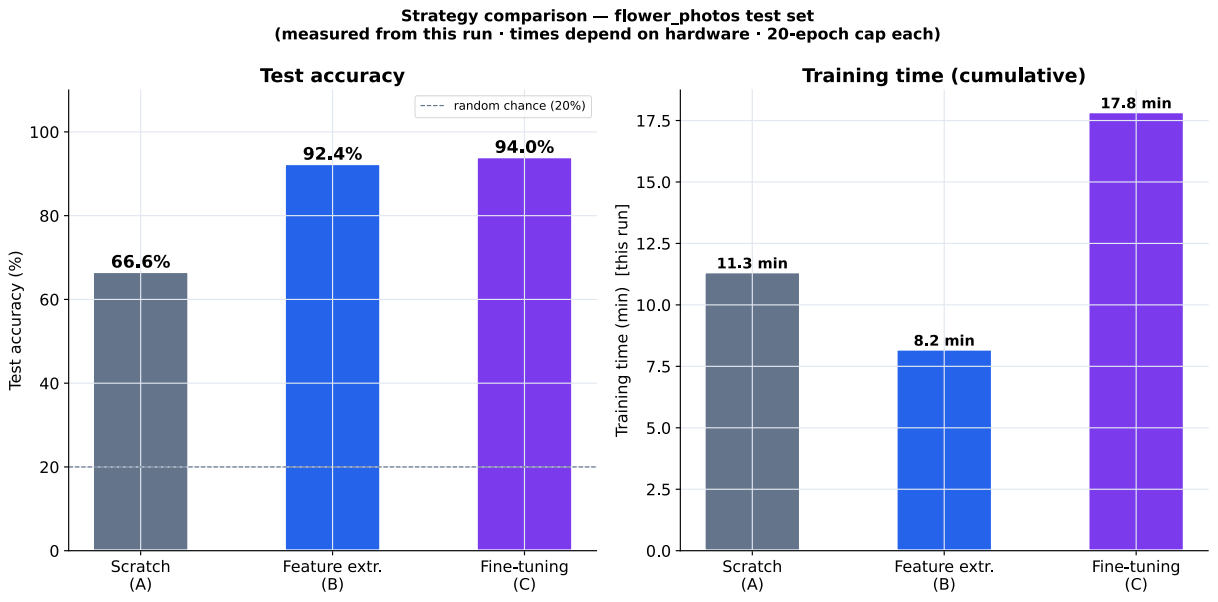
100× smaller than head LR — nudge weights gently, don't destroy features.

Notebook §4.1–§4.3 · patience=6 / epochs=20 + ReduceLROnPlateau match the notebook configuration

Results — Strategy Comparison

Section 05

Measured from this run. Feature extraction already captures most of the accuracy gain.



Times are measured from this run; absolute runtime depends on hardware and Colab availability.

A: Scratch

66.6 %

Baseline to beat — 11.3 min

B: Feature extr.

92.4 %

+25.8 pp over A — 8.2 min · the big jump

C: Fine-tuning

94.0 %

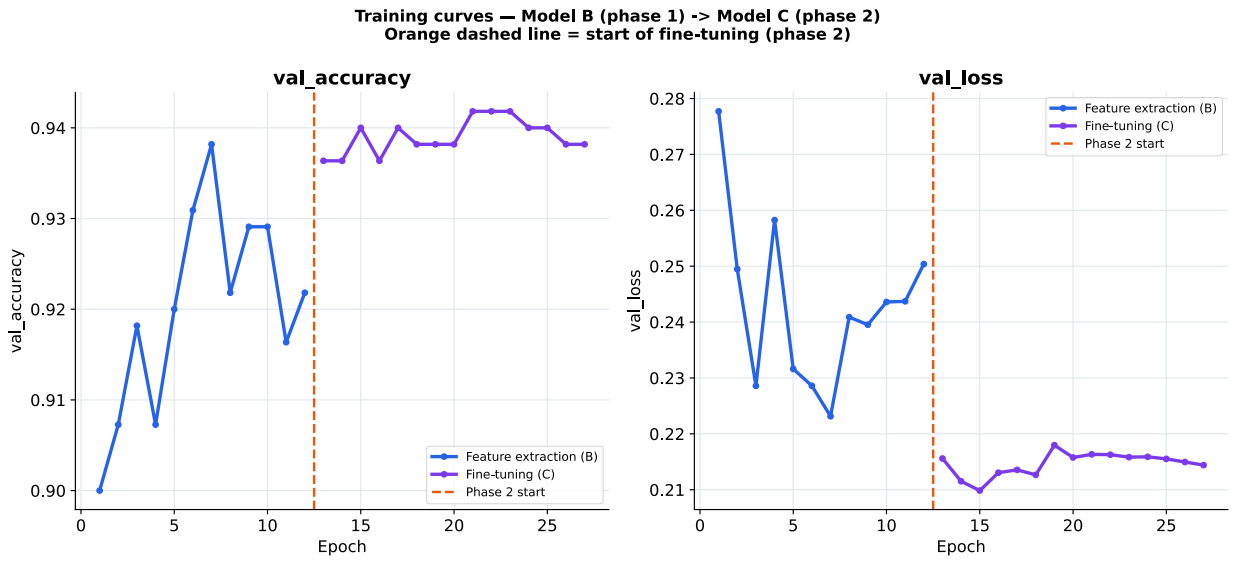
+1.6 pp over B — 17.8 min cumulative

Feature extraction does the heavy lifting (+25.8 pp). Fine-tuning adds a smaller +1.6 pp at extra cost.

Results — Training Curves

Section 05

Measured from this notebook run; vertical dashed line marks start of fine-tuning.



Curves vary across runs. Watch val_loss after unfreezing — a sustained rise means LR or unfreeze depth is too aggressive.

Phase 1 — Feature extr.

val_accuracy climbs to ~0.92 in ~10 epochs. Stable and fast.

Transition

After unfreezing, val_loss remains controlled and val_acc improves slowly. This supports the small-LR fine-tuning strategy.

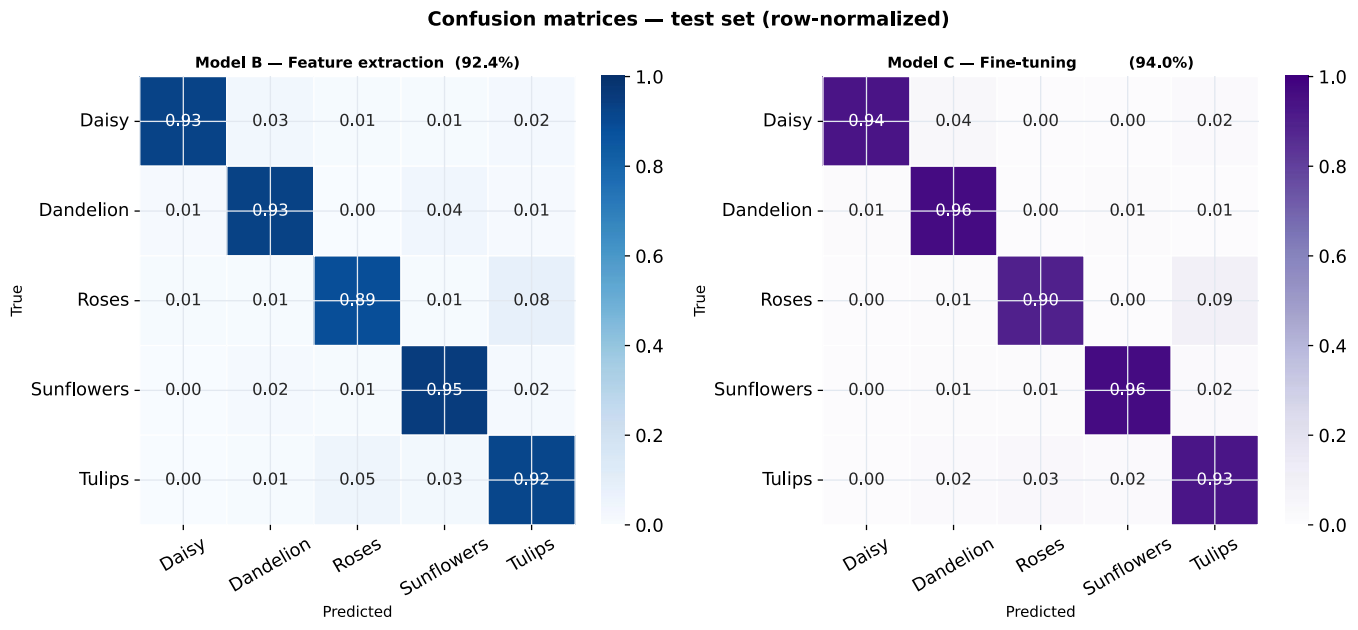
Phase 2 — Fine-tuning

Slow, controlled gain to ~0.94. The smaller LR keeps gradients tame.

Error Analysis — Confusion Matrices

Section 05

Which classes does each model confuse? Row-normalized — diagonal = recall per class.



From notebook §6 · test set, row-normalized so each row sums to 1.0

Model B

High recall on most classes — EfficientNetB0 features transfer well.

Model C

Small gains across classes, most visible for Dandelion; Roses / Tulips remain harder.

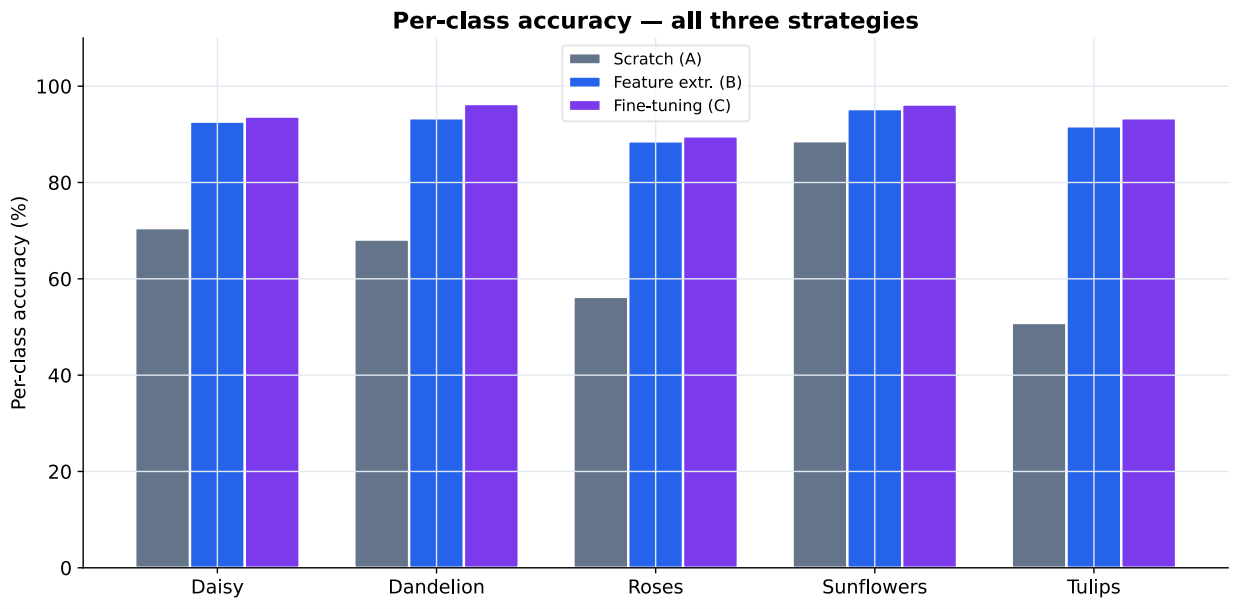
Persistent confusion

Roses ↔ Tulips remains in both models — more data or augmentation may help more than fine-tuning.

Where Does Fine-Tuning Help?

Section 05

Per-class accuracy across the three strategies — improvement is not uniform.



Insight: fine-tuning does not improve all classes equally — look at per-class metrics, not just overall accuracy

A — Scratch

Largest class-level variance — limited generalization on a small dataset.

B — Feature extraction

Strong across all classes; most of the per-class gain happens here.

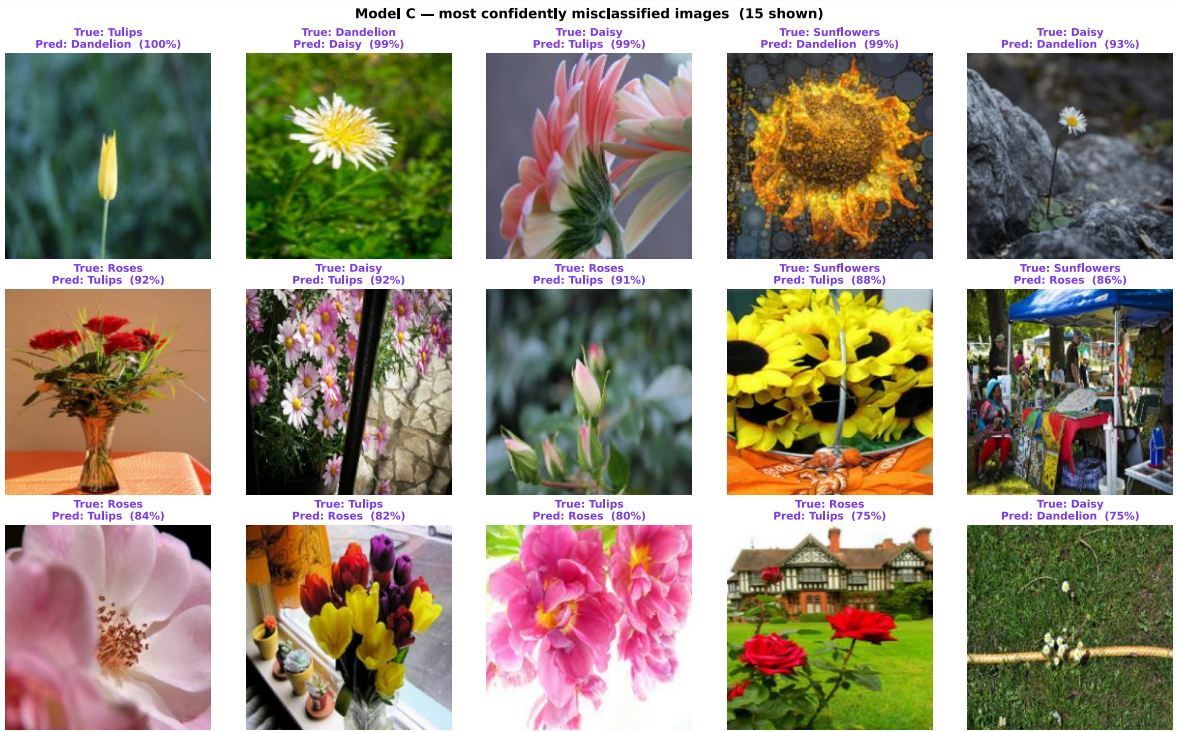
C — Fine-tuning

Adds smaller class-specific gains; the largest visible gain in this run is for Dandelion, while Roses/Tulips remain comparatively harder.

What Do the Mistakes Look Like?

Section 05

A handful of misclassified examples from Model C — error analysis at the image level.



Common pattern

High-confidence errors are mostly plausible: similar colors, cropped flowers, clutter, or close class pairs.

Why fine-tuning helps less here

These confusions reflect genuine visual similarity — extra data or hard-example mining would help more than deeper fine-tuning.

Practical action

Inspect mistakes regularly. Per-class metrics are not enough — image-level inspection often reveals labelling issues or hard cases.

Notebook §6.5 · small qualitative gallery — not a substitute for proper per-class metrics

Practical Pitfalls — Checklist

Section 05

Mistakes that cost hours in real projects.

01

Wrong input preprocessing

EfficientNetB0 keeps [0,255] (built-in rescaling). Other backbones (ResNet, VGG) need preprocess_input(). Always match the family.

02

Wrong input size

Resizing to a non-canonical size (e.g. 32×32 instead of 224×224) hurts transfer performance or breaks spatial assumptions.

03

Forgetting to recompile

Changing trainable flags without model.compile() — the optimizer still references stale variables. Always recompile.

04

LR too large for fine-tuning

Using LR=1e-3 in phase 2 destroys pretrained features in the first fine-tune epoch. Drop LR by 10–100×.

05

Unfreezing too many layers

Opening too many layers on a small dataset → severe overfit. Start shallow; unfreeze more only if val gains justify.

06

BatchNorm in training mode

Passing training=True to the base during extraction. BN running stats drift, causing unstable training.

For fair benchmarking, match training budgets across strategies. In this teaching demo, budgets differ by design — report time and epochs explicitly.

Quick Quiz

Recap

60 seconds with your neighbor — no peeking at the next slide.

You have 3,670 flower images (5 classes). You use EfficientNetB0 pretrained on ImageNet.

You unfreeze **ALL** layers and train with LR = **1e-3** from epoch 1. After 10 epochs, val_accuracy = 32% (worse than feature extraction).

What is the MOST LIKELY cause?

A The learning rate 1e-3 is too small for fine-tuning.

B Unfreezing all layers with a large LR caused catastrophic forgetting of pretrained features.

C EfficientNetB0 is too complex for a small dataset — a simpler backbone would help.

D The head was not trained first — feature extraction should be skipped entirely.

Summary · Lecture 9

Wrap-up

Feature extraction first; fine-tune only if the validation gain pays for the cost.

01	**Why transfer learning works**	ImageNet training forces a network to learn reusable visual features — edges, textures, parts — that generalize across domains.
02	**Always start with feature extraction**	Freeze the base, train only the head first. This gives a strong baseline quickly and prevents corrupting pretrained weights.
03	**Fine-tune carefully — phase by phase**	Unfreeze top layers only after the head has converged. Use LR 10–100× smaller. Recompile after every change.
04	**BatchNorm and domain gap are the big caveats**	Keep training=False on the base during extraction. Domain gap determines how deep you can fine-tune.
05	**Measure accuracy and training time together**	Fine-tuning can improve accuracy, but usually costs more training time. Match the strategy to your compute budget.

Lecture 10 preview Object detection — sliding windows, anchors, YOLO and R-CNN families. Géron Ch. 14, Object Detection section.